

Relational Model

The relational model is today the primary data model for commercial data processing applications. It attained its primary position because of its simplicity, which eases the job of the programmer.

Structure of Relational Databases

A relational database consists of a collection of tables, each of which is assigned a unique name.

For example, consider the instructor table of Figure 2.1, which stores information about instructors. The table has four column headers: *ID*, *name*, *dept name*, and *salary*. Each row of this table records information about an instructor, consisting of the instructor's *ID*, *name*, *dept name*, and *salary*.

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 2.1 The *instructor* relation.

Similarly, the [course](#) table of Figure 2.2 stores information about courses, consisting of a [course id](#), [title](#), [dept name](#), and [credits](#), for each course. Note that each instructor is identified by the value of the column [ID](#), while [each course is identified by the value of the column course_id](#).

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

Figure 2.2 The *course* relation.

Figure 2.3 shows a [third table](#), [prereq](#), which [stores the prerequisite courses for each course](#). The table has [two columns](#), [course id](#) and [prereq id](#). Each row consists of a pair of course identifiers such that the [second course is a prerequisite for the first course](#).

<i>course_id</i>	<i>prereq_id</i>
BIO-301	BIO-101
BIO-399	BIO-101
CS-190	CS-101
CS-315	CS-101
CS-319	CS-101
CS-347	CS-101
EE-181	PHY-101

Figure 2.3 The *prereq* relation.

Thus, a row in the *prereq* table indicates that two courses are *related* in the sense that one course is a prerequisite for the other. As **another example**, we consider the table *instructor*, **a row in the table can be thought of as representing the relationship between a specified ID and the corresponding values for name, dept name, and salary values.**

In general, a **row** in a table represents a *relationship* among a set of values. Since a **table** is a collection of such *relationships*.

Thus, in the **relational model the term relation is used to refer to a table**, while the term **tuple is used to refer to a row**. Similarly, the **term attribute refers to a column of a table**.

For each **attribute** of a relation, there is a set of permitted **values**, called the **domain** of that attribute. **Example**, the **domain of the salary attribute** of the *instructor* relation is the set of all possible salary values, while the **domain of the name attribute** is the set of all possible instructor names.

The **null** value is a special value that signifies that the value is unknown or does not exist. For **example**, suppose as before that we include the attribute *phone number* in the *instructor* relation. It may be that an instructor does not have a phone number at all, or that the telephone number is unlisted.

What are Keys in DBMS?

KEYS in DBMS is an attribute or sets of attributes which helps you to identify a row (tuple) in a relation (table). They allow you to find the relation between two tables. Keys help you uniquely identify a row in a table by a combination of one or more columns in that table.

Example:

Employee ID	FirstName	LastName
11	Andrew	Johnson
22	Tom	Wood
33	Alex	Hale

In the above-given example, **employee ID** is a **primary key** because it uniquely identifies an employee record. In this table, no other employee can have the same employee ID.

Why we need a Key?

Here are some reasons for using SQL keys in a DBMS.

- Keys help you to identify any row of data in a table. In a real-world application, a table could contain thousands of records. Moreover, the records could be duplicated. Keys ensure that you can uniquely identify a table record despite these challenges.
- Allows you to establish a relationship between and identify the relation between tables
- Help you to enforce identity and integrity in the relationship.

Types of Keys in Database Management System

There are mainly seven different types of Keys in DBMS, and each key has its different functionality:

1. Candidate Key: The minimal set of attributes that can uniquely identify a tuple is known as a candidate key. For Example, STUD_NO in STUDENT relation.

- It is a minimal super key.
- It is a super key with no repeated data is called a candidate key.
- The minimal set of attributes that can uniquely identify a record.

- It must contain unique values.
- It cannot contain NULL values.

• Every table must have at least a single candidate key.

• A table can have multiple candidate keys but only one primary key.

- The value of the Candidate Key is unique and may be null for a tuple.
- There can be more than one candidate key in a relationship.

Simple definition

Candidate Key = Minimal Super Key

Here, **minimal** means that we cannot remove any attribute from the key and still uniquely identify a record.

Example:

STUD_NO is the candidate key for relation STUDENT.

Table STUDENT

STUD_NO	SNAME	ADDRESS	PHONE
1	Shyam	Delhi	1E+08
2	Rakesh	Kolkata	2E+08
3	Suraj	Delhi	2E+08

- The candidate key can be simple (having only one attribute) or composite as well.

Example:

{**STUD_NO**, **COURSE_NO**} is a composite

candidate key for relation STUDENT_COURSE.

Table STUDENT_COURSE

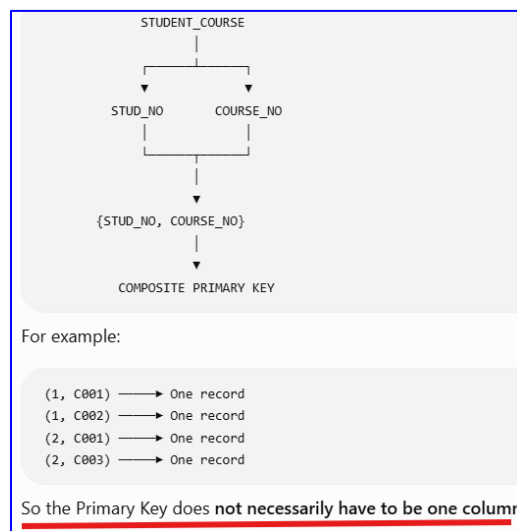
STUD_NO	TEACHER_NO	COURSE_NO
1	1	C001
2	56	C005

2. Primary Key:

Primary Key = One Candidate Key selected by the database designer to uniquely identify each record.

There can be more than one candidate key in relation out of which one can be chosen as the primary key. For Example, STUD_NO, as well as STUD_PHONE, are candidate keys for relation STUDENT but STUD_NO can be chosen as the primary key (only one out of many candidate keys).

- It is a **unique** key.
- It can **identify only one tuple (a record) at a time**.
- It has **no duplicate values**, it has unique values.
- It **cannot be NULL**.
- Primary keys are not necessarily to be a single column; **more than one column can also be a primary key** for a table. (This is called a **Composite Primary Key**.)



Example:

STUDENT table:

Student(**STUD_NO**, SNAME, ADDRESS, PHONE)

STUD_NO is a primary key

Table STUDENT

STUD_NO	SNAME	ADDRESS	PHONE
1	Shyam	Delhi	123456789
2	Rakesh	Kolkata	223365796
3	Suraj	Delhi	175468965

3. Super Key: The set of attributes that can uniquely identify a tuple is known as Super Key. For Example, STUD_NO, (STUD_NO, STUD_NAME), etc. A super key is a group of single or multiple keys that identifies rows in a table.

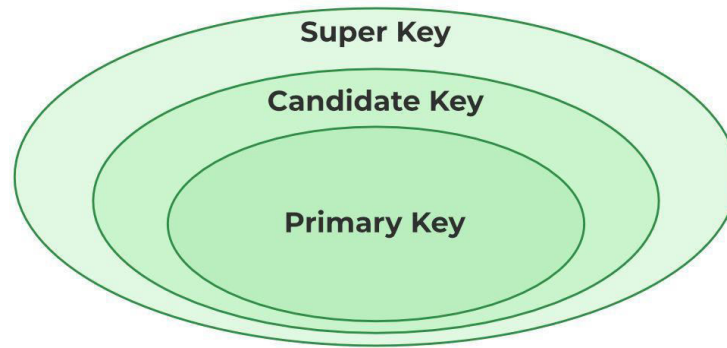
- Adding zero or more attributes to the candidate key generates the super key.

- A candidate key is a super key but vice versa is not true.
- Super Key values may also be NULL.

Example:

Consider the table shown above.

STUD_NO+PHONE is a super key.



Relation between Primary Key, Candidate Key, and Super Key

every Primary Key is a Candidate Key, and every Candidate Key is a Super Key, **but** not all Super Keys are Candidate Keys or Primary Keys.

4. Alternate Key or Secondary Key: The candidate key other than the primary key is called an **alternate key**.

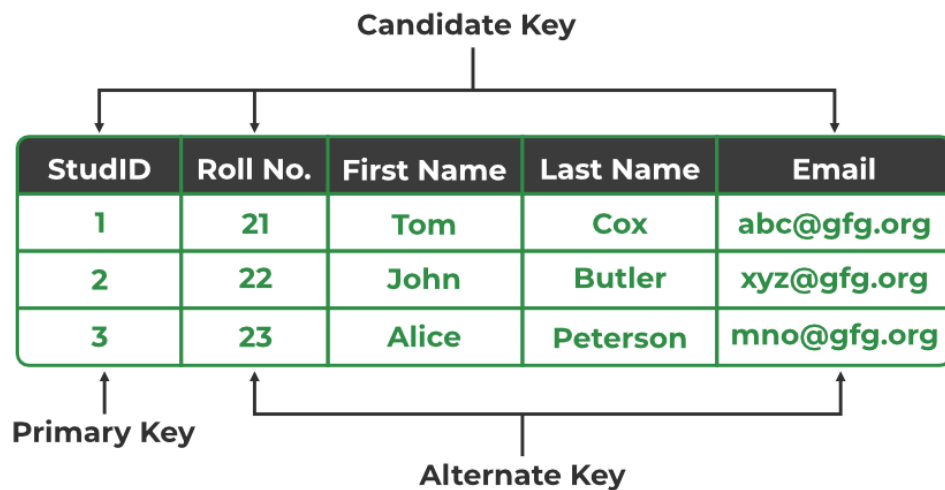
- All the keys which are not primary keys are called alternate keys.
- It is a secondary key.
- It contains two or more fields to identify two or more records.
- These values are repeated.

• Eg:- SNAME, and ADDRESS is Alternate keys

Example:

Consider the table shown above.

STUD_NO, as well as PHONE both, are candidate keys for relation STUDENT but **PHONE** will be an alternate key (only one out of many candidate keys).



Primary Key, Candidate Key, and Alternate Key

5. Foreign Key: If an attribute can only take the values which are present as values of some other attribute, it will be a foreign key to the attribute to which it refers. The relation which is being referenced is called referenced relation and the corresponding attribute is called **referenced attribute** the relation which refers to the referenced relation is called **referencing relation** and the corresponding attribute is called referencing attribute. **The referenced attribute of the referenced relation should be the primary key to it.**

- It is a key it acts as a primary key in one table and it acts as secondary key in another table.
- It combines two or more relations (tables) at a time.
- They act as a cross-reference between the tables.
- For example, DNO is a primary key in the DEPT table and a non-key in EMP

Example:

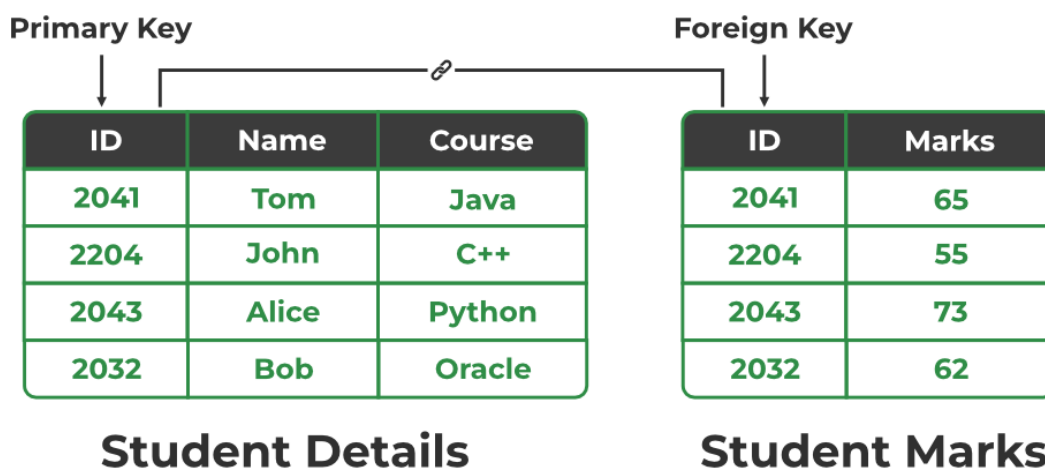
Refer Table **STUDENT** shown above.

STUD_NO in **STUDENT_COURSE** is a foreign key to **STUD_NO** in **STUDENT** relation.

Table STUDENT_COURSE

STUD_NO	TEACHER_NO	COURSE_NO
1	005	C001
2	056	C005

It may be worth noting that, **unlike** the Primary Key of any given relation, **Foreign Key** can be NULL as well as may contain duplicate tuples i.e. it need not follow uniqueness constraint. For Example, **STUD_NO** in the **STUDENT_COURSE** relation is not unique. It has been repeated for the first and third tuples. However, the **STUD_NO** in **STUDENT** relation is a primary key and it needs to be always unique, and it cannot be null.



Relation between Primary Key and Foreign Key

6. Composite Key: Sometimes, a table might not have a single column/attribute that uniquely identifies all the records of a table. To uniquely identify rows of a table, a combination of two or more columns/attributes can be used. It still can give duplicate values in rare cases. So, we need to find the optimal set of attributes that can uniquely identify rows in a table.

- It acts as a primary key if there is no primary key in a table

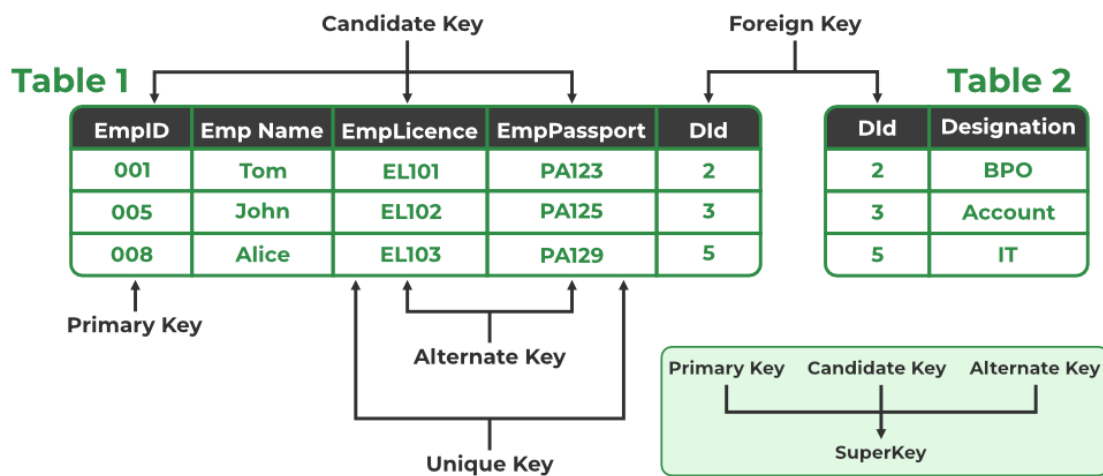
- Two or more attributes are used together to make a composite key.

- Different combinations of attributes may give different accuracy in terms of identifying the rows uniquely.

Example:

FULLNAME + DOB can be combined

together to access the details of a student.



Different Types of Keys

Composite Key
↑

student_id	subject_id	marks	exam_name

Score Table - To save scores of the student for various subjects.

In the above picture we have a **Score** table which stores the marks scored by a student in a particular subject.

In this table student_id and subject_id together will form the primary key, hence it is a composite key.

OR

A composite key is a combination of two or more columns in a table that can be used to uniquely identify each row in the table when the columns are combined uniqueness is guaranteed, but when it taken individually it does not guarantee uniqueness.

Sometimes more than one attributes are needed to uniquely identify an entity. A primary key that is made by the combination of more than one attribute is known as a **composite key**.

In other words we can say that:

Composite key is a key which is the combination of more than one field or column of a given table. It may be a candidate key or primary key.

Columns that make up the composite key can be of different data types.

```
CREATE TABLE SAMPLE_TABLE
(COL1 integer,
COL2 varchar(30),
COL3 varchar(50),
PRIMARY KEY (COL1, COL2));
```

7. Artificial key

The key created using arbitrarily assigned data are known as artificial keys. These keys are created when a primary key is large and complex and has no relationship with many other relations. The data values of the artificial keys are usually numbered in a serial order.

Example-1: If Name+PAN No. is used as a Primary Key, then it will be complex to use a such primary key. Thus, a new attribute called Row_Id can be assigned as the Primary Key. It doesn't exist in the database but it can be used to make the process simpler.

Example-2: the primary key, which is composed of Emp_ID, Emp_role, and Proj_ID, is large in employee relations. So it would be better to add a new virtual attribute to identify each tuple in the relation uniquely.

Example-3: A common example is an auto-incremented integer column (like id), which is assigned by the system.

referential integrity

Table: STUDENT

ROLL_ NO	NAME	ADDRE SS	PHONE	AGE	BRANCH_CODE(FK)
1	RAM	DELHI	94551234	18	CS

2	RAMESH	GURGAON	9652431543	18	CS
3	SUJIT	ROHTAK	9156253131	20	ECE
4	SURESH	DELHI		18	IT

Table: BRANCH

BRANCH_CODE(PK)	BRANCH_NAME
CS	COMPUTER SCIENCE
IT	INFORMATION TECHNOLOGY
ECE	ELECTRONICS AND COMMUNICATION ENGINEERING
CV	CIVIL ENGINEERING

BRANCH_CODE of STUDENT can only take the values which are present in BRANCH_CODE of BRANCH which is called referential integrity constraint.

The relation which is referencing to other relation is called **REFERENCING RELATION** (STUDENT in this case) and the relation to which other relations refer is called **REFERENCED RELATION** (BRANCH in this case).

A database schema, along with *primary key* and *foreign key* dependencies, can be depicted by **schema diagrams**. Figure 2.8 shows the schema diagram for our university organization. Each **relation (table)** appears as a **box**, with the relation name **at the top in blue**, and the **attributes listed inside the box**. **Primary key** attributes are shown **underlined**. **Foreign key dependencies** appear as **arrows** from the foreign key attributes of the referencing relation to the primary key of the referenced relation.

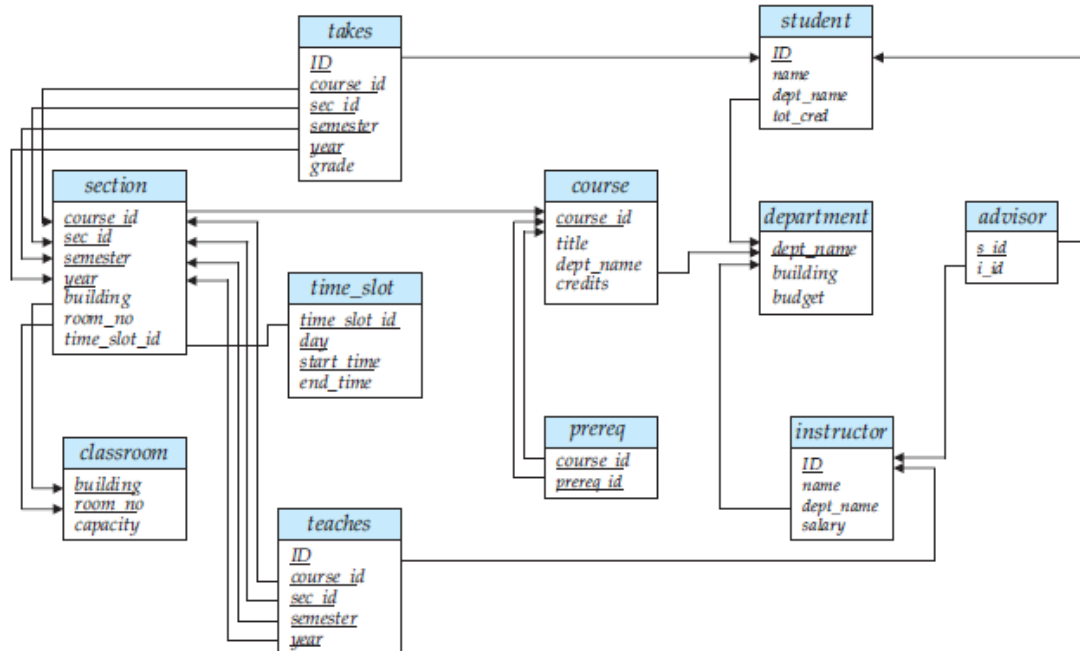


Figure 2.8 Schema diagram for the university database.

Referential integrity constraints other than foreign key constraints are not shown explicitly in schema diagrams.

Relational Query Languages

A **query language** is a language in which a user requests information from the database. These languages are usually on a level higher than that of a standard programming language.

Query languages can be categorized as either **procedural** or **nonprocedural**. In a **procedural language**, the user instructs the system to perform a sequence of operations on the database to compute the desired result.

Examples of Procedural languages:

FORTRAN, COBOL, ALGOL, BASIC, C and Pascal.

In a **nonprocedural language**, the user describes the desired information without giving a specific procedure for obtaining that information.

Examples of Non-Procedural languages:

SQL, PROLOG, LISP.